MGL 7230: Tests Logiciels

Lecture # 5

Tests d'Intégration

Based on Chapter 11 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson, 2013.

Plan

- Introduction
- Objectif des tests d'intégration
- Quand les tests d’intégration ont-ils lieu ?
- Décisions à prendre avant les tests d'intégration
- Erreurs d'intégration
- Hiérarchie d'intégration
- Meilleures pratiques en matière de tests d'intégration

Introduction

- Combinaison **de deux ou plusieurs unités logicielles**
  - Souvent un sous-ensemble du projet global
- Pourquoi les ingénieurs logiciels se soucient-ils de l'intégration ?
  - De nouveaux problèmes surgiront inévitablement
    - De nombreux systèmes, désormais réunis, qui n'existaient pas auparavant
  - Si c'est mal fait, tous les problèmes se présentent en même temps
    - Difficile à diagnostiquer, déboguer et réparer
  - Cascade d'interdépendances
    - Impossible de trouver et de résoudre les problèmes un par un

Introduction

- Un composant logiciel est parfois également appelé :
  - une classe,
  - un ensemble de classes,
  - un module,
  - un ensemble de méthodes
  - …

- L'objectif principal des tests d'intégration est de déterminer si un sous-système composé de plusieurs unités **fonctionne comme prévu..**

Objectif des tests d'intégration

- L'objectif des tests d'intégration est de **détecter les problèmes pouvant survenir lors de l'intégration des composants.**
- Les composants C1 et C2 peuvent avoir été testés et fonctionner correctement.
  - Cependant, cela ne garantit pas que, une fois intégrés, C1 et C2 fonctionneront comme prévu.
- Une erreur découverte lors des tests d'intégration est souvent appelée **erreur d'intégration**.
- Lors des tests d'intégration, il est possible de découvrir une erreur dans l'un des composants testés précédemment.
  - Cependant, les erreurs d'intégration sont celles qui ne sont susceptibles d'être révélées que lorsque les composants sont intégrés et testés ensemble.

Quand les tests d'intégration ont-ils lieu ?

- Les tests d'intégration se déroulent à différents niveaux.
  - Un développeur peut coder une classe C1 et la tester.
  - Le même développeur peut ensuite coder la classe C2 qui utilise C1 et tester C1 et C2 comme une unité intégrée.
  - Ce processus peut se poursuivre à mesure que le développeur progresse vers la création d'un composant relativement complexe composé de deux classes ou plus.
- Une équipe de testeurs peut intégrer deux ou plusieurs composants ou sous-systèmes complexes, éventuellement développés par différentes équipes au sein d'une organisation.
- Dans un autre cas, une telle équipe peut intégrer et tester plusieurs composants développés en interne avec un ou plusieurs composants provenant de sources différentes.

Décisions à prendre avant les tests d'intégration

1. ***Hiérarchie d'intégration :*** Les composants doivent-ils être intégrés de manière descendante, ascendante ou selon une approche mixte ?

2. ***Séquençage et stubbing :*** Dans quel ordre les composants doivent-ils être intégrés et quels stubs sont nécessaires ?

3. ***Génération de tests :*** Quelles méthodes utiliser pour générer les tests et mesurer leur adéquation ?

Erreurs d'intégration

- Les tests d'intégration se concentrent sur les connexions entre les composants.
  - Détectez les **erreurs** susceptibles d'entraîner une mauvaise communication entre les composants.

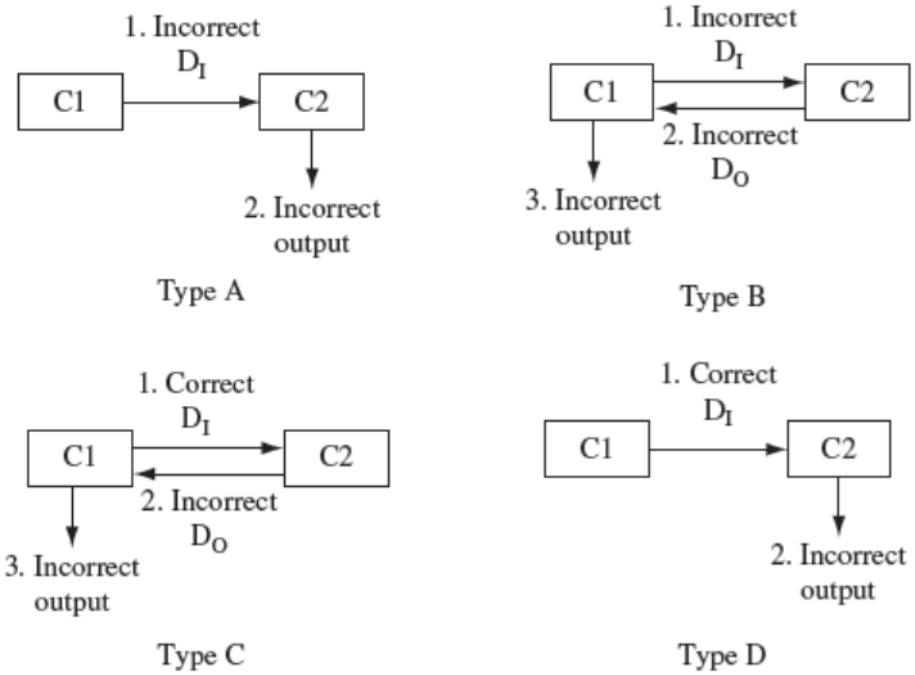
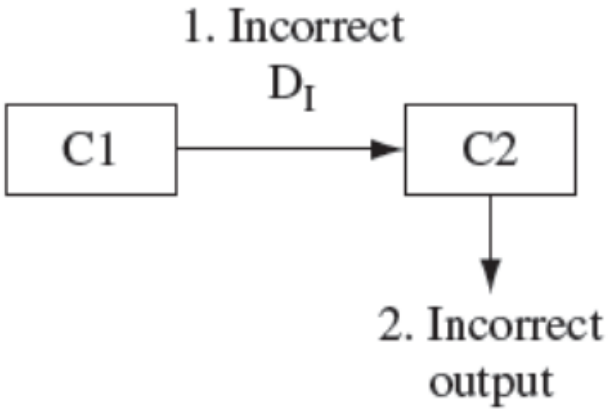


Figure 11.1 Types of integration errors. D_I is a set or a sequence of inputs passed from component C1 to component C2. D_O is a set or a sequence of outputs returned from component C2 to component C1.

Erreurs d'intégration

- **Erreurs de type A :**
  - Supposons qu'un appel de C1 à une méthode de C2 envoie des valeurs de paramètres incorrectes (événement 1).
  - À son tour, C2 calcule des résultats incorrects (événement 2).
  - Une erreur se produit dans le composant C1, et peut-être aussi dans le composant C2.



Type A

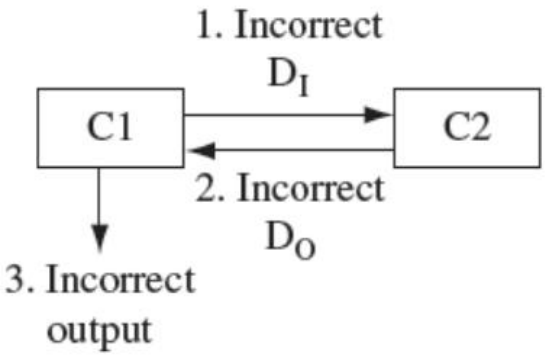
Di est un ensemble ou une séquence d'entrées passées du composant c1 au composant c2.

Erreurs d'intégration

- **Erreurs de type B**
  - Lorsque C1 reçoit un résultat incorrect de C2 (événement 2), il génère également une sortie incorrecte (événement 3).
  - C1 comporte une erreur, mais C2 peut en avoir une ou non.

Di est un ensemble ou une séquence d'entrées passées du composant c1 au composant c2.

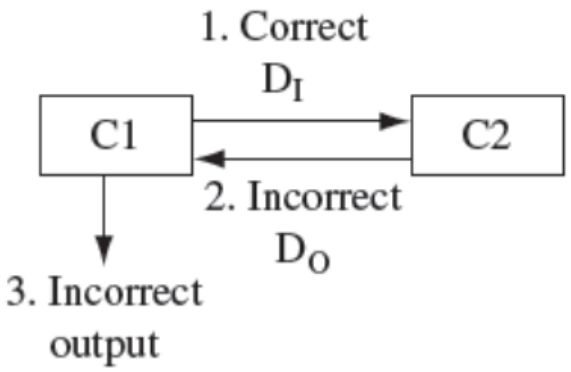
Do est un ensemble ou une séquence de sorties retournées du composant c2 vers le composant c1.



Type B

Erreurs d'intégration

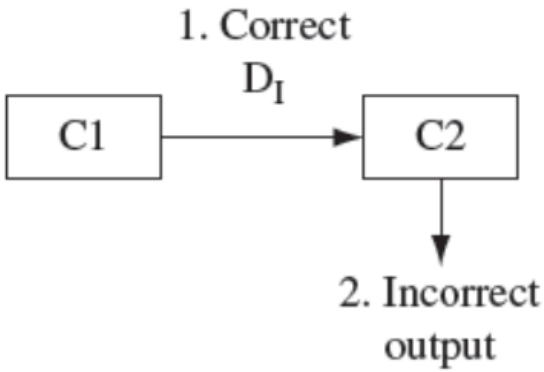
- **Erreurs de type C**
  - Peut survenir en raison de plusieurs types d'erreurs de codage entraînant une sortie incorrecte, laquelle entraîne à son tour une sortie incorrecte de C1.
  - **Erreur dans le composant C2 et peut-être aussi dans C1**



Type C

Erreurs d'intégration

- **Erreurs de type D**
  - Similaire à l'erreur de type C, sauf que la sortie incorrecte de C2 n'affecte pas C1.
  - Erreur dans C2.



Type D

Dépendance

- **Dépendance des données**
  - La classe C1 peut générer des données et les écrire dans un fichier. La classe C2 peut lire les données écrites par C1 et effectuer un traitement supplémentaire.

- **Dépendance fonctionnelle**
  - C1 pourrait utiliser une fonction de C2.

- **Dépendance de contrôle**
  - C1 pourrait évaluer une condition qui influe sur l'exécution de C2.

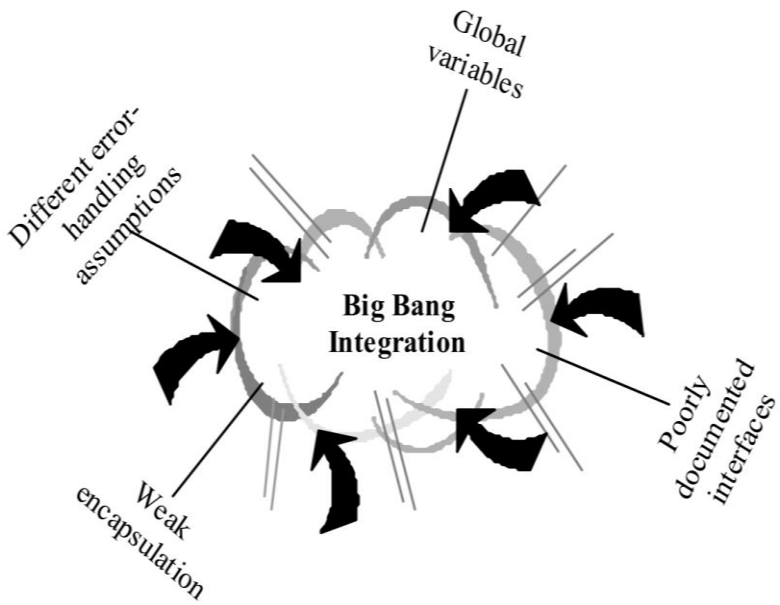
Hiérarchie d'intégration

Il existe quatre principaux types d'intégration pour les tests basés sur la décomposition structurelle :

1. Big Bang
2. Descendant
3. Ascendant
4. Combinaison (sandwich)

1. Intégration Big Bang

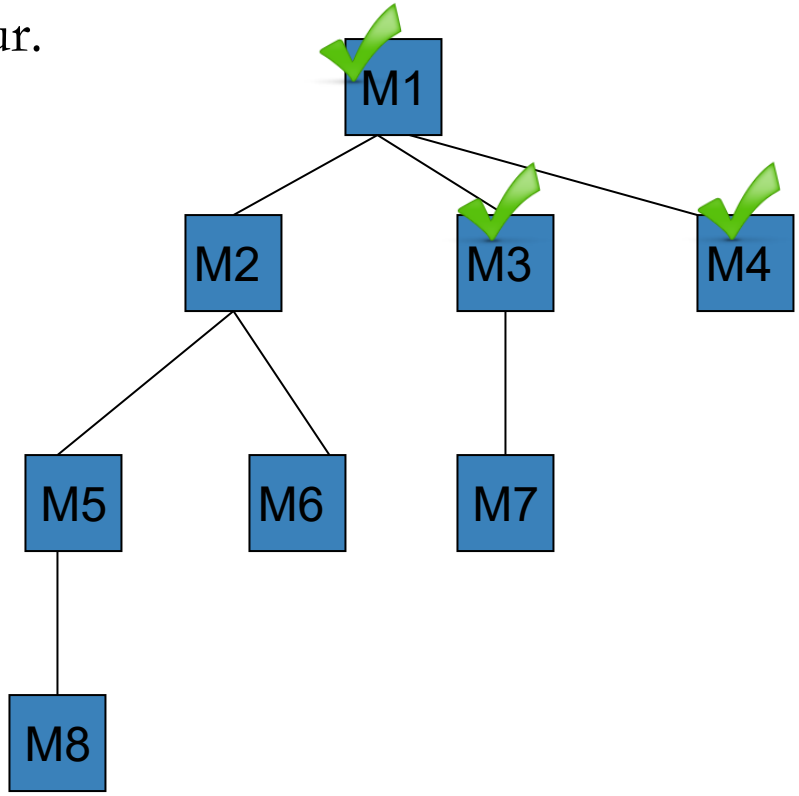
- Concevoir, coder, tester les unités, déboguer chaque classe/unité/sous-système séparément.
- Combiner le tout et tester le système.
- Prier. ☺



Avantages :
Peut convenir aux petits systèmes (mais pas aux applications d'entreprise).
Inconvénients :
- Délai : Attendre que tous les modules soient terminés et effectuer une intégration, puis tester tous les modules intégrés ensemble.
- Les erreurs sont identifiées très tardivement, et il est très difficile d'identifier la panne.
- Difficile de garantir que tous les cas de test d'intégration sont couverts.

2. Tests d'intégration descendants

- Les modules sont intégrés par ordre décroissant de la hiérarchie de contrôle.
- Les modules subordonnés au module de contrôle principal sont intégrés par ordre décroissant de largeur.
- De haut en bas :
  - M1, stub(M2), stub(M3), stub(M4)
  - M1, M4, stub(M2), stub(M3)
  - M1, M4, stub(M2), M3, stub(M7)
  - M1, M4, stub(M2), M3, M7
  - M1,M2, M3, M4, stub(M5), stub(M6)
  - …



Étapes des tests d'intégration descendants

- Le module principal est utilisé comme pilote de test.
- Des stubs remplacent tous les composants directement subordonnés au module de contrôle principal.
- Les stubs sont remplacés un par un par des composants réels.
- Des tests sont effectués au fur et à mesure de l'intégration de chaque composant.

Avantages et inconvénients des tests d'intégration descendants

Avantages

- Les cas de test peuvent être définis en fonction des fonctionnalités du système (exigences fonctionnelles).
- Aucun driver requis.

Inconvénients

- **L'écriture de stubs est complexe**: ils doivent permettre de tester toutes les conditions possibles.
- Un grand nombre de stubs peut être nécessaire, surtout si le niveau le plus bas du système contient de nombreuses méthodes.

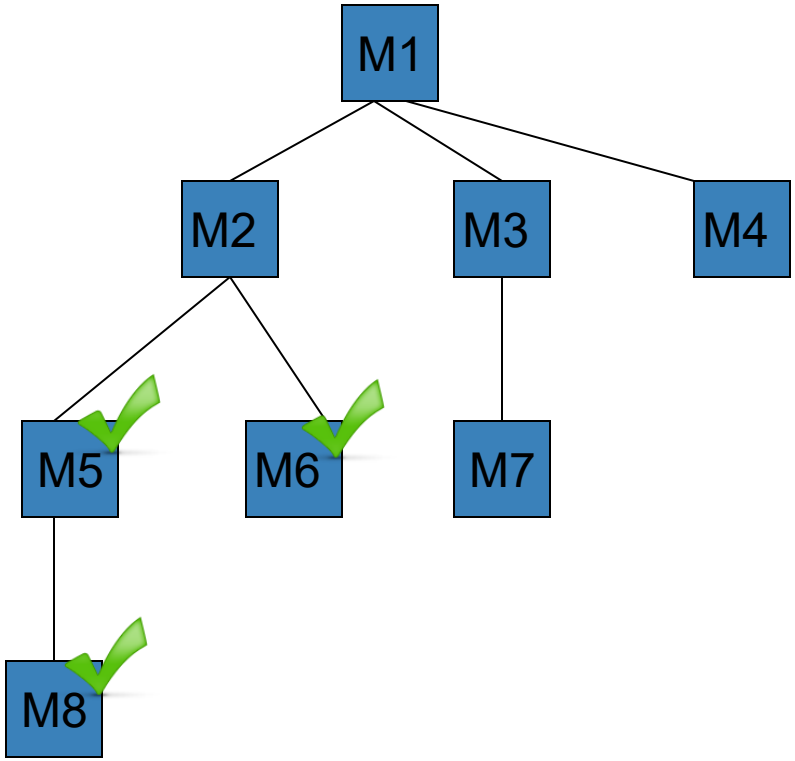
3. Intégration ascendante

- Les composants de bas niveau sont regroupés en clusters.
- Un pilote est écrit pour coordonner les entrées et les sorties des cas de test.
- Le cluster est testé.
- Les pilotes sont supprimés et les clusters sont regroupés en remontant dans la structure du programme.

Tests d'intégration ascendants

De bas en haut :

- M8, driver(M5)
- M5, M8, driver(M2)
- M6, driver(M2)
- M5, M8, M6, driver(M2)
- …



Avantages et inconvénients des tests d'intégration ascendants

- **Avantages**
  - Pas besoin de stubs
  - Modules opérationnels testés en profondeur
  - Utile pour les tests d'intégration des systèmes suivants
    - Systèmes orientés objet
    - Systèmes temps réel (par exemple, systèmes avec des exigences strictes de performance).

- **Inconvenients**
  - Teste l'interface utilisateur en dernier
  - Drivers nécessaires

4. Sandwich Integration

- Combinaison d'intégrations ascendantes et descendantes
- Le système est considéré comme des couches
- **Approche 1 :**
  - L'approche descendante est utilisée pour la couche supérieure
  - Une approche ascendante est utilisée pour la couche inférieure
  - Permet de commencer l'intégration tôt dans la phase de test

- **Approche 2 :**
  - Commencez par une couche au milieu
  - Utiliser les pilotes et les ébauches
  - Permet de commencer l'intégration tôt dans la phase de test

Avantages et inconvénients des tests sandwich

- **Avantages :**
  - Les tests des couches supérieure et inférieure peuvent être effectués en parallèle.
  - Utile pour les très grands projets comportant des sous-projets.

- **Inconvénients :**
  - L'approche « big-bang » est utilisée pour la couche intermédiaire.
  - Compétences de test différentes (descendantes et ascendantes).
  - Ne convient pas aux petits systèmes avec une forte interdépendance entre les différents modules.

Choisir la stratégie d'intégration

Cela dépend de plusieurs facteurs, notamment :

- La disponibilité du code
- La difficulté de construction des stubs et des pilotes
- La taille et l'expérience de l'équipe de test
- La disponibilité des outils

Comparaison de la stratégie d'intégration

- Seules les approches Big Bang ne nécessitent pas de stubs ni de pilotes.
  - Les approches descendantes et ascendantes nécessitent toutes deux des stubs et des pilotes.
- L'approche **descendante** présente l'avantage de pouvoir disposer d'un système partiellement fonctionnel avant l'intégration complète (lorsque la direction souhaite présenter un système à un groupe restreint de clients potentiels).
- L'approche **ascendante** construit le système de bas en haut ; un système fonctionnel n'est donc disponible qu'une fois les composants de niveau supérieur intégrés.
- Dans la plupart des systèmes modernes utilisant une interface utilisateur graphique (GUI), il peut être préférable de commencer l'intégration dans la GUI.
  - Assurez-vous qu'un système squelettique est toujours disponible, que l'approche soit descendante ou ascendante.

Meilleures pratiques en matière de tests d'intégration

- Commencez par déterminer **la stratégie de test** d'intégration à adopter (création de couches), puis préparez les cas de test et les données de test en conséquence.
- Étudiez la conception architecturale de l'application et identifiez les modules critiques (modules à haut risque). **Ceux-ci sont prioritaires.**
- Obtenez les conceptions d'interface auprès de l'équipe d'architecture et créez des cas de test pour vérifier toutes les interfaces en détail. L'interface avec la base de données, le matériel externe ou l'application logicielle doit être testée en détail.
- Concentrez-vous sur les données de test (entrées/sorties), qui jouent un rôle essentiel.